Use Notebook Instances

An *Amazon SageMaker notebook instance* is a fully managed ML compute instance running the Jupyter Notebook App. Amazon SageMaker manages creating the instance and related resources. Use Jupyter notebooks in your notebook instance to prepare and process data, write code to train models, deploy models to Amazon SageMaker hosting, and test or validate your models.

Topics

- Create a Notebook Instance (p. 36)
- Access Notebook Instances (p. 39)
- Customize a Notebook Instance (p. 40)
- Use Example Notebooks (p. 42)
- Notebook Instance Software Updates (p. 44)
- Set the Notebook Kernel (p. 44)
- Install External Libraries and Kernels in Notebook Instances (p. 45)
- Associate Git Repositories with Amazon SageMaker Notebook Instances (p. 46)
- Get Notebook Instance Metadata (p. 54)
- Monitor Jupyter Logs in Amazon CloudWatch Logs (p. 54)

Create a Notebook Instance

To create a notebook instance, use either the Amazon SageMaker console or the CreateNotebookInstance (p. 656) API.

After receiving the request, Amazon SageMaker does the following:

- **Creates a network interface**—If you choose the optional VPC configuration, it creates the network interface in your VPC. It uses the subnet ID that you provide in the request to determine which Availability Zone to create the subnet in. Amazon SageMaker associates the security group that you provide in the request with the subnet. For more information, see Connect a Notebook Instance to Resources in a VPC (p. 516).
- **Launches an ML compute instance**—Amazon SageMaker launches an ML compute instance in an Amazon SageMaker VPC. Amazon SageMaker performs the configuration tasks that allow it to manage your notebook instance, and if you specified your VPC, it enables traffic between your VPC and the notebook instance.
- **Installs Anaconda packages and libraries for common deep learning platforms**—Amazon SageMaker installs all of the Anaconda packages that are included in the installer. For more information, see Anaconda package list. In addition, Amazon SageMaker installs the TensorFlow and Apache MXNet deep learning libraries.
- **Attaches an ML storage volume**—Amazon SageMaker attaches an ML storage volume to the ML compute instance. You can use the volume to clean up the training dataset or to temporarily store other data to work with. Choose any size between 5 GB and 16384 GB, in 1 GB increments, for the volume. The default is 5 GB. ML storage volumes are encrypted, so Amazon SageMaker can't determine the amount of available free space on the volume. Because of this, you can increase the volume size when you update a notebook instance, but you can't decrease the volume size. If you want to decrease the size of the ML storage volume in use, create a new notebook instance with the desired size.

Important
Only files and data saved within the `/home/ec2-user/SageMaker` folder persist between notebook instance sessions. Files and data that are saved outside this directory are overwritten when the notebook instance stops and restarts.

Note
Each notebook instance's /tmp directory provides a minimum of 10 GB of storage in an instant store. An instance store is temporary, block-level storage that isn't persistent. When the instance is stopped or restarted, Amazon SageMaker deletes the directory's contents. This temporary storage is part of the root volume of the notebook instance.

- **Copies example Jupyter notebooks**— These Python code examples illustrate model training and hosting exercises using various algorithms and training datasets.

To create a notebook instance:

1. Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker/.
2. Choose **Notebook instances**, then choose **Create notebook instance**.
3. On the **Create notebook instance** page, provide the following information:
   a. For **Notebook instance name**, type a name for your notebook instance.
   b. For **Instance type**, choose an instance type for your notebook instance. For a list of supported instance types, see Amazon SageMaker Limits.
   c. For **Elastic Inference**, choose an inference accelerator type to associate with the notebook instance, or choose **none**. For information about elastic inference, see Amazon SageMaker Elastic Inference (EI) (p. 355).
   d. For **IAM role**, choose either an existing IAM role in your account that has the necessary permissions to access Amazon SageMaker resources or **Create a new role**. If you choose **Create a new role**, for **Create an IAM role**:
      i. If you want to use S3 buckets other than the one you created in Step 1: Create an Amazon S3 Bucket (p. 17) to store your input data and output, choose them.

         The IAM role automatically has permissions to use any bucket that has `sagemaker` as part of its name. The `AmazonSageMakerFullAccess` policy, which Amazon SageMaker attaches to the role, gives the role those permissions.

         To give access to other S3 buckets from your notebook instance

         - If you're not concerned about users in your AWS account accessing your data, choose **Any S3 bucket**.
         - If your account has sensitive data (such as Human Resources information), restrict access to certain buckets by choosing **Specific S3 buckets**. You can update the permissions policy attached to the role you are creating later.
         - To explicitly control access, restrict access by choosing **None**. Use bucket and object names and tags as supported by the `AmazonSageMakerFullAccess` policy. For more information, see AmazonSageMakerFullAccess Policy (p. 506).

      ii. Choose **Create role**.

         Amazon SageMaker creates an IAM role named `AmazonSageMaker-ExecutionRole-`*`YYYYMMDD`*`T`*`HHmmSS`*. For example, `AmazonSageMaker-ExecutionRole-20171125T090800`.

         To see the policies that are attached to the role, use the IAM console.

         Open the IAM console at https://console.aws.amazon.com/iam/.

You can see that the following policies are attached to the role:

- A trust policy that allows Amazon SageMaker to assume the role.
- The `AmazonSageMakerFullAccess` AWS managed policy.
- If you gave access to additional S3 bucket(s) when creating this role, the customer managed policy attached to the role. The name of the customer managed policy is `AmazonSageMaker-ExecutionPolicy-`*`YYYYMMDD`*`T`*`HHmmSS`*.

For more information about creating your own IAM role, see Amazon SageMaker Roles (p. 496).

e. For **Root access**, to enable root access for all notebook instance users,choose **Enabled**. To disable root access for users, choose **Disabled**.If you enable root access, all notebook instance users have administrator privileges and can access and edit all files on it.

   Note
   If you disable root access, you will still be able to set up lifecycle configurations, as described later in this procedure.

f. (Optional) Allow access to resources in your Virtual Private Cloud (VPC).

   To access resources in your VPC from the notebook instance

   i. Choose the **VPC** and a **SubnetId**.

   ii. For **Security Group**, choose your VPC's default security group. For this exercise and others in this guide) the inbound and outbound rules of the default security group are sufficient.

   iii. To allow connecting to a resource in your VPC, ensure that the resource resolves to a private IP address in your VPC. For example, to ensure that an Amazon Redshift DNS name resolves to a private IP address, do one of the following:

   - Ensure that the Amazon Redshift cluster is not publicly accessible.
   - If the Amazon Redshift cluster is publicly accessible, set the `DNS resolution` and `DNS hostnames` VPC parameters to `true`. For more information, see Managing Clusters in an Amazon Virtual Private Cloud (VPC)

   iv. By default, a notebook instance can't connect to on-premises resources or to a peer VPC. You can create a lifecycle configuration that creates an entry in your route table that enables connection to on-premises resources or to a peer VPC. For information, see Understanding Amazon SageMaker notebook instance networking configurations and advanced routing options.

g. If you allowed access to resources from your VPC, enable direct internet access. For **Direct internet access**, choose **Enable**. Without internet access, you can't train or host models from notebooks on this notebook instance unless your VPC has a NAT gateway and your security group allows outbound connections. For more information, see Connect a Notebook Instance to Resources in a VPC (p. 516).

h. (Optional) To use shell scripts that run when you create or start the instance, specify a lifecycle configuration. For information, see Customize a Notebook Instance (p. 40)

i. (Optional) If you want Amazon SageMaker to use an AWS Key Management Service (AWS KMS) key to encrypt data in the ML storage volume attached to the notebook instance, specify the key.

j. Specify the size, in GB, of the ML storage volume that is attached to the notebook instance. You can choose a size between 5 GB and 16,384 GB, in 1 GB increments. You can use the volume to clean up the training dataset when you no longer need it or to temporarily store other data to work with.

k. (Optional) To associate Git repositories with the notebook instance, choose a default repository and up to three additional repositories. For more information, see Associate Git Repositories with Amazon SageMaker Notebook Instances (p. 46).

l. Choose **Create notebook instance**.

   In a few minutes, Amazon SageMaker launches an ML compute instance—in this case, a notebook instance—and attaches an ML storage volume to it. The notebook instance has a preconfigured Jupyter notebook server and a set of Anaconda libraries. For more information, see the CreateNotebookInstance (p. 656) API.

4. When the status of the notebook instance is `InService`, choose **Open Jupyter** next to its name to open the classic Jupyter dashboard, or choose **Open JupyterLab** to open the JupyterLab dashboard. For more information, see Access Notebook Instances (p. 39).

   The dashboard provides access to:

   - Sample notebooks. Amazon SageMaker provides sample notebooks that contain complete code walkthroughs. These walkthroughs show how to use Amazon SageMaker to perform common machine learning tasks. For more information, see Use Example Notebooks (p. 42).
   - The kernels for Jupyter, including those that provide support for Python 2 and 3, Apache MXNet, TensorFlow, PySpark, and R. To create a new notebook and choose a kernel for that notebook, use the **New** menu.

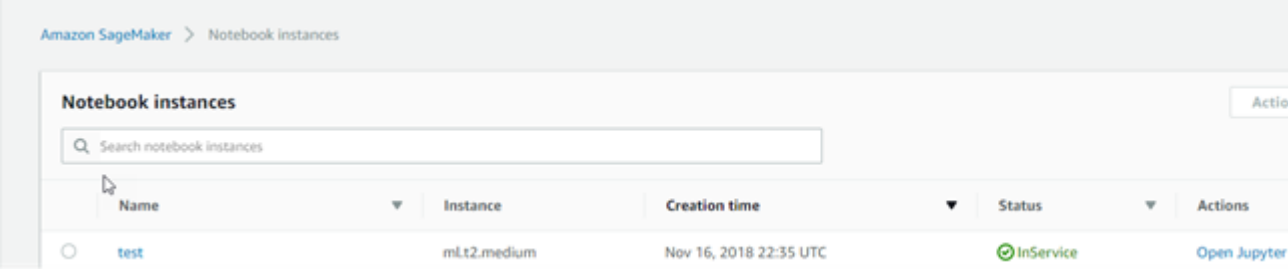
   For more information about Jupyter notebooks, see The Jupyter notebook.

Access Notebook Instances

To access your Amazon SageMaker notebook instances, choose one of the following options:

- Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker/.

  Choose **Notebook instances**. The console displays a list of notebook instances in your account. To open a notebook instance with a standard Jupyter interface, choose **Open Jupyter** for that instance. To open a notebook instance with a JupyterLab interface, choose **Open JupyterLab** for that instance.



  The console uses your sign-in credentials to send a CreatePresignedNotebookInstanceUrl (p. 665) API request to Amazon SageMaker. Amazon SageMaker returns the URL for your notebook instance, and the console opens the URL in another browser tab and displays the Jupyter notebook dashboard.

  Note
  The URL that you get from a call to CreatePresignedNotebookInstanceUrl (p. 665) is valid only for 5 minutes. If you try to use the URL after the 5-minute limit expires, you are directed to the AWS Management Console sign-in page.

- Use the API.

  To get the URL for the notebook instance, call the CreatePresignedNotebookInstanceUrl (p. 665) API and use the URL that the API returns to open the notebook instance.

Use the Jupyter notebook dashboard to create and manage notebooks and to write code. For more information about Jupyter notebooks, see http://jupyter.org/documentation.html.

Control Root Access to a Notebook Instance

By default, when you create a notebook instance, users that log into that notebook instance have root access. Data science is an iterative process that might require the data scientist to test and use different software tools and packages, so many notebook instance users need to have root access to be able to install these tools and packages. Because users with root access have administrator privileges, users can access and edit all files on a notebook instance with root access enabled.

If you don't want users to have root access to a notebook instance, when you call CreateNotebookInstance (p. 656) or UpdateNotebookInstance (p. 844) operations, set the `RootAccess` field to `Disabled`. You can also disable root access for users when you create or update a notebook instance in the Amazon SageMaker console. For information, see Step 2: Create an Amazon SageMaker Notebook Instance (p. 17).

> **Note**
> Lifecycle configurations need root access to be able to set up a notebook instance. Because of this, lifecycle configurations associated with a notebook instance always run with root access even if you disable root access for users.

Customize a Notebook Instance

To install packages or sample notebooks on your notebook instance, configure networking and security for it, or otherwise use a shell script to customize it, use a lifecycle configuration. A *lifecycle configuration* provides shell scripts that run only when you create the notebook instance or whenever you start one. When you create a notebook instance, you can create a new lifecycle configuration and the scripts it uses or apply one that you already have.

The Amazon SageMaker team maintains a public repository of notebook intance lifecycle configurations that address common use cases for customizing notebook instances at https://github.com/aws-samples/amazon-sagemaker-notebook-instance-lifecycle-configuration-samples.

> **Note**
> Each script has a limit of 16384 characters.
> The value of the `$PATH` environment variable that is available to both scripts is `/usr/local/sbin:/usr/local/bin:/usr/bin:/usr/sbin:/sbin:/bin`. The working directory, which is the value of the `$PWD` environment variable, is `/`.
> View CloudWatch Logs for notebook instance lifecycle configurations in log group `/aws/sagemaker/NotebookInstances` in log stream `[notebook-instance-name]/[LifecycleConfigHook]`.
> Scripts cannot run for longer than 5 minutes. If a script runs for longer than 5 minutes, it fails and the notebook instance is not created or started. To help decrease the run time of scripts, try the following:
>
> - Cut down on necessary steps. For example, limit which conda environments in which to install large packages.
> - Run tasks in parallel processes.
> - Use the `nohup` command in your script.

To create a lifecycle configuration

1. For **Lifecycle configuration - *Optional***, choose **Create a new lifecycle configuration**.

2. For **Name**, type a name.
3. (Optional) To create a script that runs when you create the notebook and every time you start it, choose **Start notebook**.
4. In the **Start notebook** editor, type the script.
5. (Optional) To create a script that runs only once, when you create the notebook, choose **Create notebook**.
6. In the **Create notebook** editor, type the script configure networking.
7. Choose **Create configuration**.

You can see a list of notebook instance lifecycle configurations you previously created by choosing **Lifecycle configuration** in the Amazon SageMaker console. From there, you can view, edit, delete existing lifecycle configurations. You can create a new notebook instance lifecycle configuration by choosing **Create configuration**. These notebook instance lifecycle configurations are available when you create a new notebook instance.

Lifecycle Configuration Best Practices

The following are best practices for using lifecycle configurations:

- Lifecycle configurations run as the `root` user. If your script makes any changes within the `/home/ec2-user/SageMaker` directory, (for example, installing a package with `pip`), use the command `sudo -u ec2-user` command to run as the `ec2-user` user. This is the same user that Amazon SageMaker runs as.
- Amazon SageMaker notebook instances use `conda` environments to implement different kernels for Jupyter notebooks. If you want to install packages that are available to one or more notebook kernels, enclose the commands to install the packages with `conda` environment commands that activate the conda environment that contains the kernel where you want to install the packages.

  For example, if you want to install a package only in for the `python3` environment, use the following code:

```
#!/bin/bash
sudo -u ec2-user -i <<'EOF'

# This will affect only the Jupyter kernel called "conda_python3".
source activate python3

# Replace myPackage with the name of the package you want to install.
pip install myPackage
# You can also perform "conda install" here as well.

source deactivate

EOF
```

  If you want to install a package in all conda environments in the notebook instance, use the following code:

```
#!/bin/bash
sudo -u ec2-user -i <<'EOF'

# Note that "base" is special environment name, include it there as well.
for env in base /home/ec2-user/anaconda3/envs/*; do
    source /home/ec2-user/anaconda3/bin/activate $(basename "$env")

    # Installing packages in the Jupyter system environment can affect stability of your
 SageMaker
```

```
    # Notebook Instance.  You can remove this check if you'd like to install Jupyter
 extensions, etc.
    if [ $env = 'JupyterSystemEnv' ]; then
      continue
    fi

    # Replace myPackage with the name of the package you want to install.
    pip install --upgrade --quiet myPackage
    # You can also perform "conda install" here as well.

    source /home/ec2-user/anaconda3/bin/deactivate
done

EOF
```

Important
When you create or change a script file, we recommend you use **Create notebook** editor or a text editor that allows for Unix style line breaks. Copying text from a non Linux operating system might include incompatible line breaks and result in an unexpected error.

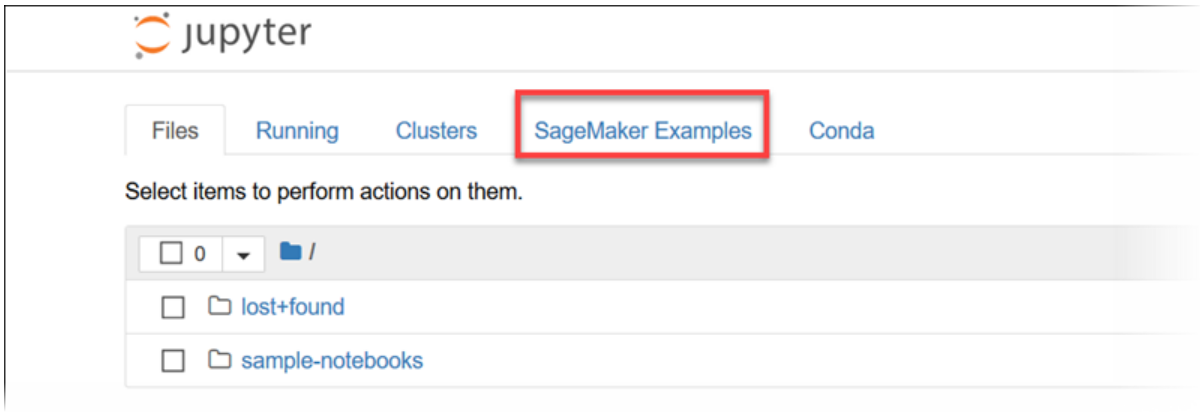
Use Example Notebooks

Your notebook instance contains example notebooks provided by Amazon SageMaker. The example notebooks contain code that shows how to apply machine learning solutions by using Amazon SageMaker. Notebook instances use the `nbexamples` Jupyter extension, which enables you to view a read-only version of an example notebook or create a copy of it so that you can modify and run it. For more information about the `nbexamples` extension, see https://github.com/danielballan/nbexamples.

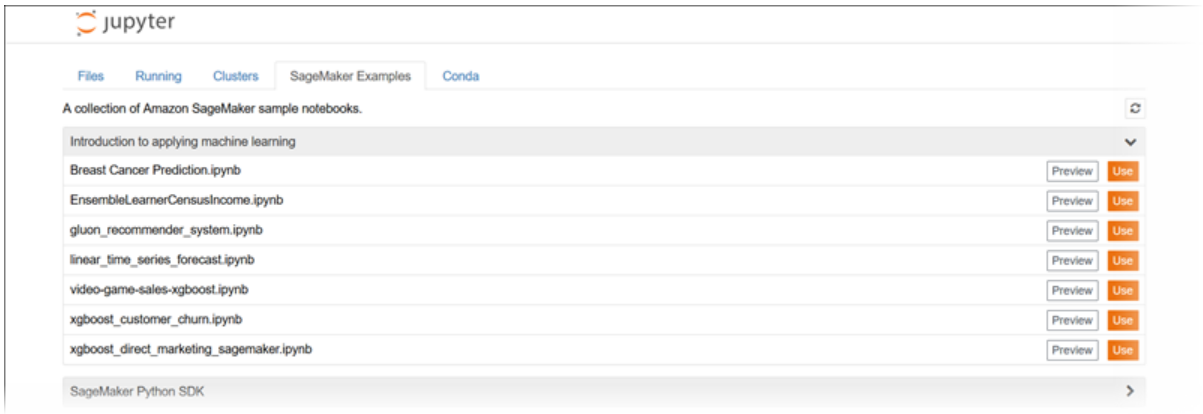
Note
Example notebooks typically download datasets from the internet. If you disable Amazon SageMaker-provided internet access when you create you notebook instance, example notebooks might not work. For more information, see Connect a Notebook Instance to Resources in a VPC (p. 516).

Use or View Example Notebooks in Jupyter Classic

To view or use the example notebooks in the classic Jupyter view, choose the **SageMaker Examples** tab.

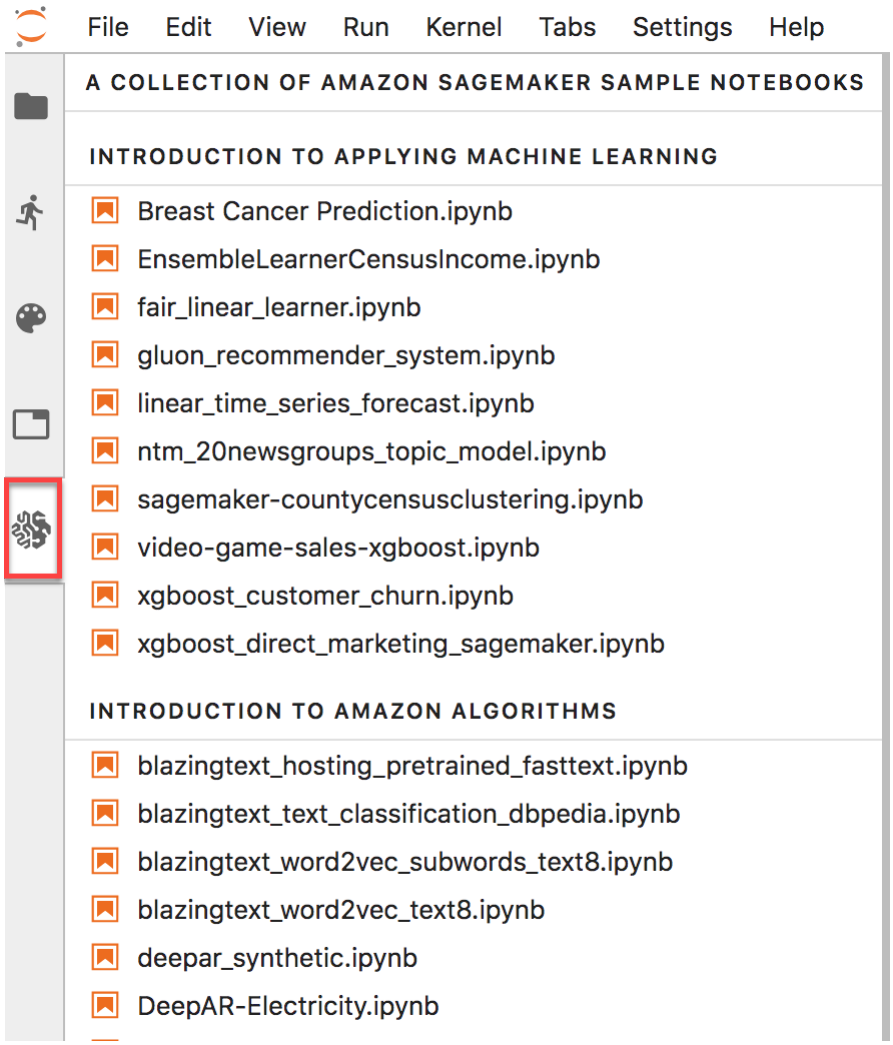


To view a read-only version of an example notebook in the Jupyter classic view, on the **SageMaker Examples** tab, choose **Preview** for that notebook. To create a copy of an example notebook in the home directory of your notebook instance, choose **Use**. In the dialog box, you can change the notebook's name before saving it.



Use or View Example Notebooks in Jupyterlab

To view or use the example notebooks in the Jupyterlab view, choose the examples icon in the left navigation panel.



To view a read-only version of an example notebook, choose the name of the notebook. This opens the notebook as a tab in the main area. To create a copy of an example notebook in the home directory of your notebook instance, choose **Create a Copy** in the top banner. In the dialog box, type a name for the notebook and then choose **CREATE COPY**.

For more information about the example notebooks, see the Amazon SageMaker examples GitHub repository.

Notebook Instance Software Updates

Amazon SageMaker periodically tests and releases software that is installed on notebook instances. This includes:

- Kernel updates
- Security patches
- AWS SDK updates
- Amazon SageMaker Python SDK updates
- Open source software updates

Amazon SageMaker does not automatically update software on a notebook instance when it is in service. To ensure that you have the most recent software updates, stop and restart your notebook instance, either in the Amazon SageMaker console or by calling StopNotebookInstance (p. 832) followd by StartNotebookInstance (p. 824).

You can also manually update software installed on your notebook instance while it is running by using update commands in a terminal or in a notebook.

> **Note**
> Updating kernels and some packages might depend on whether root access is enabled for the notebook instance. For more information, see Control Root Access to a Notebook Instance (p. 40).

Notebook instances do not notify you if you are running outdated software. You can check the Personal Health Dashboard or the security bulletin at https://aws.amazon.com/security/security-bulletins/ for updates.

Set the Notebook Kernel

Amazon SageMaker provides several kernels for Jupyter that provide support for Python 2 and 3, Apache MXNet, TensorFlow, and PySpark. To set a kernel for a new notebook in the Jupyter notebook dashboard, choose **New**, and then choose the kernel from the list.

Install External Libraries and Kernels in Notebook Instances

Amazon SageMaker notebook instances come with multiple environments already installed. These environments contain Jupyter kernels and Python packages including: scikit, Pandas, NumPy, TensorFlow, and MXNet. These environments, along with all files in the `sample-notebooks` folder, are refreshed when you stop and start a notebook instance. You can also install your own environments that contain your choice of packages and kernels. This is typically done using `conda install` or `pip install`.

The different Jupyter kernels in Amazon SageMaker notebook instances are separate conda environments. For information about conda environments, see Managing environments in the *Conda* documentation. If you want to use an external library in a specific kernel, install the library in the environment for that kernel. You can do this either in the terminal or in a notebook cell. The following procedures show how to install Theano so that you can use it in a notebook with a conda_mxnet_p36 kernel.

To install Theano from a terminal

1. Open a notebook instance.
2. In the Jupyter dashboard, choose **New**, and then choose **Terminal**.
3. In the terminal, type the following commands:

```
conda install -n mxnet_p36 -c conda-forge theano
   python
   import theano
```

To install Theano from a Jupyter notebook cell

1. Open a notebook instance.
2. In the Jupyter dashboard, choose **New**, and then choose **conda_mxnet_p36**.
3. In a cell in the new notebook, type the following command:

```
!pip install theano
```

Maintain a Sandboxed Python Environment

Amazon SageMaker periodically updates the Python and dependency versions in the environments installed on a notebook instance when it is stopped and restarted. For more information, see Notebook Instance Software Updates (p. 44). To maintain an isolated Python environment that does not change versions, create a lifecycle configuration that runs each time you start your notebook instance. For information about creating lifecycle configurations, see Customize a Notebook Instance (p. 40).

The following example lifecycle configuration script installs Miniconda on your notebook instance. This allows you to create environments in your notebook instance with specific versions of Python and dependencies that Amazon SageMaker does not update:

```
#!/bin/bash

set -e

WORKING_DIR=/home/ec2-user/.myproject
```

```
mkdir -p "$WORKING_DIR"

# Install Miniconda to get a separate python and pip
wget https://repo.anaconda.com/miniconda/Miniconda3-4.5.12-Linux-x86_64.sh -O
 "$WORKING_DIR/miniconda.sh"

# Install Miniconda into the working directory
bash "$WORKING_DIR/miniconda.sh" -b -u -p "$WORKING_DIR/miniconda"

# Install pinned versions of any dependencies
source "$WORKING_DIR/miniconda/bin/activate"
pip install boto3==1.9.86
pip install requests==2.21.0

# Bootstrapping code

# Cleanup
source "$WORKING_DIR/miniconda/bin/deactivate"
rm -rf "$WORKING_DIR/miniconda.sh"
```

You can also add a sandboxed Python installation as a kernel that you can use in a Jupyter notebook by including the following code to the above lifecycle configuration:

```
source "$WORKING_DIR/miniconda/bin/activate"

# If required, add this as a kernel
pip install ipykernel
python -m ipykernel install --user --name MyProjectEnv --display-name "Python
 (myprojectenv)"

source "$WORKING_DIR/miniconda/bin/deactivate"
```

Associate Git Repositories with Amazon SageMaker Notebook Instances

Associate Git repositories with your notebook instance to save your notebooks in a source control environment that persists even if you stop or delete your notebook instance. You can associate one default repository and up to three additional repositories with a notebook instance. The repositories can be hosted in AWS CodeCommit, GitHub or on any other Git server. Associating Git repositories with your notebook instance can be useful for:

- Persistence - Notebooks in a notebook instance are stored on durable Amazon EBS volumes, but they do not persist beyond the life of your notebook instance. Storing notebooks in a Git repository enables you to store and use notebooks even if you stop or delete your notebook instance.
- Collaboration - Peers on a team often work on machine learning projects together. Storing your notebooks in Git repositories allows peers working in different notebook instances to share notebooks and collaborate on them in a source-control environment.
- Learning - Many Jupyter notebooks that demonstrate machine learning techniques are available in publicly hosted Git repositories, such as on GitHub. You can associate your notebook instance with a repository to easily load Jupyter notebooks contained in that repository.

There are two ways to associate a Git repository with a notebook instance:

- Add a Git repository as a resource in your Amazon SageMaker account. Then, to access the repository, you can specify an AWS Secrets Manager secret that contains credentials. That way, you can access repositories that require authentication.

- Associate a public Git repository that is not a resource in your account. If you do this, you cannot specify credentials to access the repository.

Topics

- Add a Git Repository to Your Amazon SageMaker Account (p. 47)
- Create a Notebook Instance with an Associated Git Repository (p. 49)
- Associate a CodeCommit Repository in a Different AWS Account with a Notebook Instance (p. 51)
- Use Git Repositories in a Notebook Instance (p. 52)

Add a Git Repository to Your Amazon SageMaker Account

To manage your GitHub repositories, easily associate them with your notebook instances, and associate credentials for repositories that require authentication, add the repositories as resources in your Amazon SageMaker account. You can view a list of repositories that are stored in your account and details about each repository in the Amazon SageMaker console and by using the API.

You can add Git repositories to your Amazon SageMaker account in the Amazon SageMaker console or by using the AWS CLI.

> **Note**
> You can use the Amazon SageMaker API CreateCodeRepository (p. 627) to add Git repositories to your Amazon SageMaker account, but step-by-step instructions are not provided here.

Add a Git Repository to Your Amazon SageMaker Account (Console)

To add a Git repository as a resource in your Amazon SageMaker account

1. Open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker/.
2. Choose **Git repositories**, then choose **Add repository**.
3. To add an CodeCommit repository, choose **AWS CodeCommit**.
   a. To use an existing CodeCommit repository:
      i. Choose **Use existing repository**.
      ii. For **Repository**, choose a repository from the list.
      iii. Enter a name to use for the repository in Amazon SageMaker. The name must be 1 to 63 characters. Valid characters are a-z, A-Z, 0-9, and - (hyphen).
      iv. Choose **Add repository**.
   b. To create a new CodeCommit repository:
      i. Choose **Create new repository**.
      ii. Enter a name for the repository that you can use in both CodeCommit and Amazon SageMaker. The name must be 1 to 63 characters. Valid characters are a-z, A-Z, 0-9, and - (hyphen).
      iii. Choose **Create repository**.
4. To add a Git repository hosted somewhere other than CodeCommit :
   a. Choose **GitHub/Other Git-based repo**.
   b. Enter a name to use for the repository in Amazon SageMaker. The name must be 1 to 63 characters. Valid characters are a-z, A-Z, 0-9, and - (hyphen).

c. Enter the URL for the repository.

   Note
   Do not provide a user name in the URL. Add the username and password in AWS Secrets Manager as described in the next step.

d. For **Git credentials**, choose the credentials to use to authenticate to the repository. This is necessary only if the Git repository is private.

   Note
   If you have two-factor authentication enabled for your Git repository, use a personal access token generated by your Git service provider instead of a password.

   i. To use an existing AWS Secrets Manager secret, choose **Use existing secret**, and then choose a secret from the list. For information about creating and storing a secret, see Creating a Basic Secret in the *AWS Secrets Manager User Guide*. The name of the secret you use must contain the string `sagemaker`.

      Note
      The secret must have a staging label of `AWSCURRENT` and must be in the following format:
      `{"username": UserName, "password": Password}`
      For GitHub repositories, we recommend using a personal access token instead of your account password. For information, see https://help.github.com/articles/creating-a-personal-access-token-for-the-command-line/.

   ii. To create a new AWS Secrets Manager secret, choose **Create secret**, enter a name for the secret, and then enter the username and password to use to authenticate to the repository. The name for the secret must contain the string `sagemaker`.

      Note
      The IAM role you use to create the secret must have the `secretsmanager:GetSecretValue` permission in its IAM policy.
      The secret must have a staging label of `AWSCURRENT` and must be in the following format:
      `{"username": UserName, "password": Password}`
      For GitHub repositories, we recommend using a personal access token instead of your account password.

   iii. To not use any credentials, choose **No secret**.

e. Choose **Create secret**.

Add a Git Repository to Your Amazon SageMaker Account (CLI)

Use the `create-code-repository` AWS CLI command. Specify a name for the repository as the value of the `code-repository-name` argument. The name must be 1 to 63 characters. Valid characters are a-z, A-Z, 0-9, and - (hyphen). Also specify the following:

- The default branch
- The URL of the Git repository

  Note
  Do not provide a user name in the URL. Add the username and password in AWS Secrets Manager as described in the next step.

- The Amazon Resource Name (ARN) of an AWS Secrets Manager secret that contains the credentials to use to authenticate the repository as the value of the `git-config` argument

For information about creating and storing a secret, see Creating a Basic Secret in the *AWS Secrets Manager User Guide*. The following command creates a new repository named `MyRespository` in

your Amazon SageMaker account that points to a Git repository hosted at `https://github.com/myprofile/my-repo"`.

For Linux, OS X, or Unix:

```
aws sagemaker create-code-repository \
                    --code-repository-name "MyRepository" \
                    --git-config '{"Branch":"master", "RepositoryUrl" :
                    "https://github.com/myprofile/my-repo", "SecretArn" :
 "arn:aws:secretsmanager:us-east-2:012345678901:secret:my-secret-ABc0DE"}'
```

For Windows:

```
aws sagemaker create-code-repository ^
                    --code-repository-name "MyRepository" ^
                    --git-config "{\"Branch\":\"master\", \"RepositoryUrl\" :
                    \"https://github.com/myprofile/my-repo\", \"SecretArn\" :
 \"arn:aws:secretsmanager:us-east-2:012345678901:secret:my-secret-ABc0DE\"}"
```

> **Note**
> The secret must have a staging label of `AWSCURRENT` and must be in the following format:
> `{"username": UserName, "password": Password}`
> For GitHub repositories, we recommend using a personal access token instead of your account password.

Create a Notebook Instance with an Associated Git Repository

You can associate Git repositories with a notebook instance when you create the notebook instance by using the AWS Management Console, or the AWS CLI.

> **Note**
> You can use the Amazon SageMaker API CreateNotebookInstance (p. 656) to associate Git repositories with a notebook instance, but step-by-step instructions are not provided here.

> **Note**
> If you want to use a CodeCommit repository that is in a different AWS than the notebook instance,set up cross-account access for the repository. For information, see Associate a CodeCommit Repository in a Different AWS Account with a Notebook Instance (p. 51).

Topics

- Create a Notebook Instance with an Associated Git Repository (Console) (p. 49)
- Create a Notebook Instance with an Associated Git Repository (CLI) (p. 50)

Create a Notebook Instance with an Associated Git Repository (Console)

To create a notebook instance and associate Git repositories in the AWS Management Console

1. Follow the instructions at Step 2: Create an Amazon SageMaker Notebook Instance (p. 17).
2. For **Git repositories**, choose Git repositories to associate with the notebook instance.

   a. For **Default repository**, choose a repository that you want to use as your default repository. Amazon SageMaker clones this repository as a subdirectory in the Jupyter startup directory

at `/home/ec2-user/SageMaker`. When you open your notebook instance, it opens in this repository. To choose a repository that is stored as a resource in your account, choose its name from the list. To add a new repository as a resource in your account, choose **Add a repository to Amazon SageMaker (opens the Add repository flow in a new window)** and then follow the instructions at Create a Notebook Instance with an Associated Git Repository (Console) (p. 49). To clone a public repository that is not stored in your account, choose **Clone a public Git repository to this notebook instance only**, and then specify the URL for that repository.

b. For **Additional repository 1**, choose a repository that you want to add as an additional directory. Amazon SageMaker clones this repository as a subdirectory in the Jupyter startup directory at `/home/ec2-user/SageMaker`. To choose a repository that is stored as a resource in your account, choose its name from the list. To add a new repository as a resource in your account, choose **Add a repository to Amazon SageMaker (opens the Add repository flow in a new window)** and then follow the instructions at Create a Notebook Instance with an Associated Git Repository (Console) (p. 49). To clone a repository that is not stored in your account, choose **Clone a public Git repository to this notebook instance only**, and then specify the URL for that repository.

   Repeat this step up to three times to add up to three additional repositories to your notebook instance.

Create a Notebook Instance with an Associated Git Repository (CLI)

To create a notebook instance and associate Git repositories by using the AWS CLI, use the `create-notebook-instance` command as follows:

- Specify the repository that you want to use as your default repository as the value of the `default-code-repository` argument. Amazon SageMaker clones this repository as a subdirectory in the Jupyter startup directory at `/home/ec2-user/SageMaker`. When you open your notebook instance, it opens in this repository. To use a repository that is stored as a resource in your Amazon SageMaker account, specify the name of the repository as the value of the `default-code-repository` argument. To use a repository that is not stored in your account, specify the URL of the repository as the value of the `default-code-repository` argument.
- Specify up to three additional repositories as the value of the `additional-code-repositories` argument. Amazon SageMaker clones this repository as a subdirectory in the Jupyter startup directory at `/home/ec2-user/SageMaker`, and the repository is excluded from the default repository by adding it to the `.git/info/exclude` directory of the default repository. To use repositories that are stored as resources in your Amazon SageMaker account, specify the names of the repositories as the value of the `additional-code-repositories` argument. To use repositories that are not stored in your account, specify the URLs of the repositories as the value of the `additional-code-repositories` argument.

For example, the following command creates a notebook instance that has a repository named `MyGitRepo`, that is stored as a resource in your Amazon SageMaker account, as a default repository, and an additional repository that is hosted on GitHub:

```
aws sagemaker create-notebook-instance \
                    --notebook-instance-name "MyNotebookInstance" \
                    --instance-type "ml.t2.medium" \
                    --role-arn "arn:aws:iam::012345678901:role/service-role/
AmazonSageMaker-ExecutionRole-20181129T121390" \
                    --default-code-repository "MyGitRepo" \
                    --additional-code-repositories "https://github.com/myprofile/my-other-
repo"
```

Note
If you use an AWS CodeCommit repository that does not contain "SageMaker" in its name, add the `codecommit:GitPull` and `codecommit:GitPush` permissions to the role that you pass as the `role-arn` argument to the `create-notebook-instance` command. For information about how to add permissions to a role, see Adding and Removing IAM Policies in the *AWS Identity and Access Management User Guide*.

Associate a CodeCommit Repository in a Different AWS Account with a Notebook Instance

To associate a CodeCommit repository in a different AWS account with your notebook instance, set up cross-account access for the CodeCommit repository.

To set up cross-account access for a CodeCommit repository and associate it with a notebook instance:

1. In the AWS account that contains the CodeCommit repository, create an IAM policy that allows access to the repository from users in the account that contains your notebook instance. For information, see Step 1: Create a Policy for Repository Access in AccountA in the *CodeCommit User Guide*.
2. In the AWS account that contains the CodeCommit repository, create an IAM role, and attach the policy that you created in the previous step to that role. For information, see Step 2: Create a Role for Repository Access in AccountA in the *CodeCommit User Guide*.
3. Create a profile in the notebook instance that uses the role that you created in the previous step:
   a. Open the notebook instance.
   b. Open a terminal in the notebook instance.
   c. Edit a new profile by typing the following in the terminal:

   ```
   vi /home/ec2-user/.aws/config
   ```

   d. Edit the file with the following profile information:

   ```
   [profile CrossAccountAccessProfile]
   region = us-west-2
   role_arn =
    arn:aws:iam::CodeCommitAccount:role/CrossAccountRepositoryContributorRole
   credential_source=Ec2InstanceMetadata
   output = json
   ```

   Where *`CodeCommitAccount`* is the account that contains the CodeCommit repository, *`CrossAccountAccessProfile`* is the name of the new profile, and *`CrossAccountRepositoryContributorRole`* is the name of the role you created in the previous step.
4. On the notebook instance, configure git to use the profile you created in the previous step:
   a. Open the notebook instance.
   b. Open a terminal in the notebook instance.
   c. Edit the Git configuration file typing the following in the terminal:

   ```
   vi /home/ec2-user/.gitconfig
   ```

   d. Edit the file with the following profile information:

```
[credential]
        helper = !aws codecommit credential-helper --
profile CrossAccountAccessProfile $@
        UseHttpPath = true
```

Where *CrossAccountAccessProfile* is the name of the profile that you created in the previous step.

Use Git Repositories in a Notebook Instance

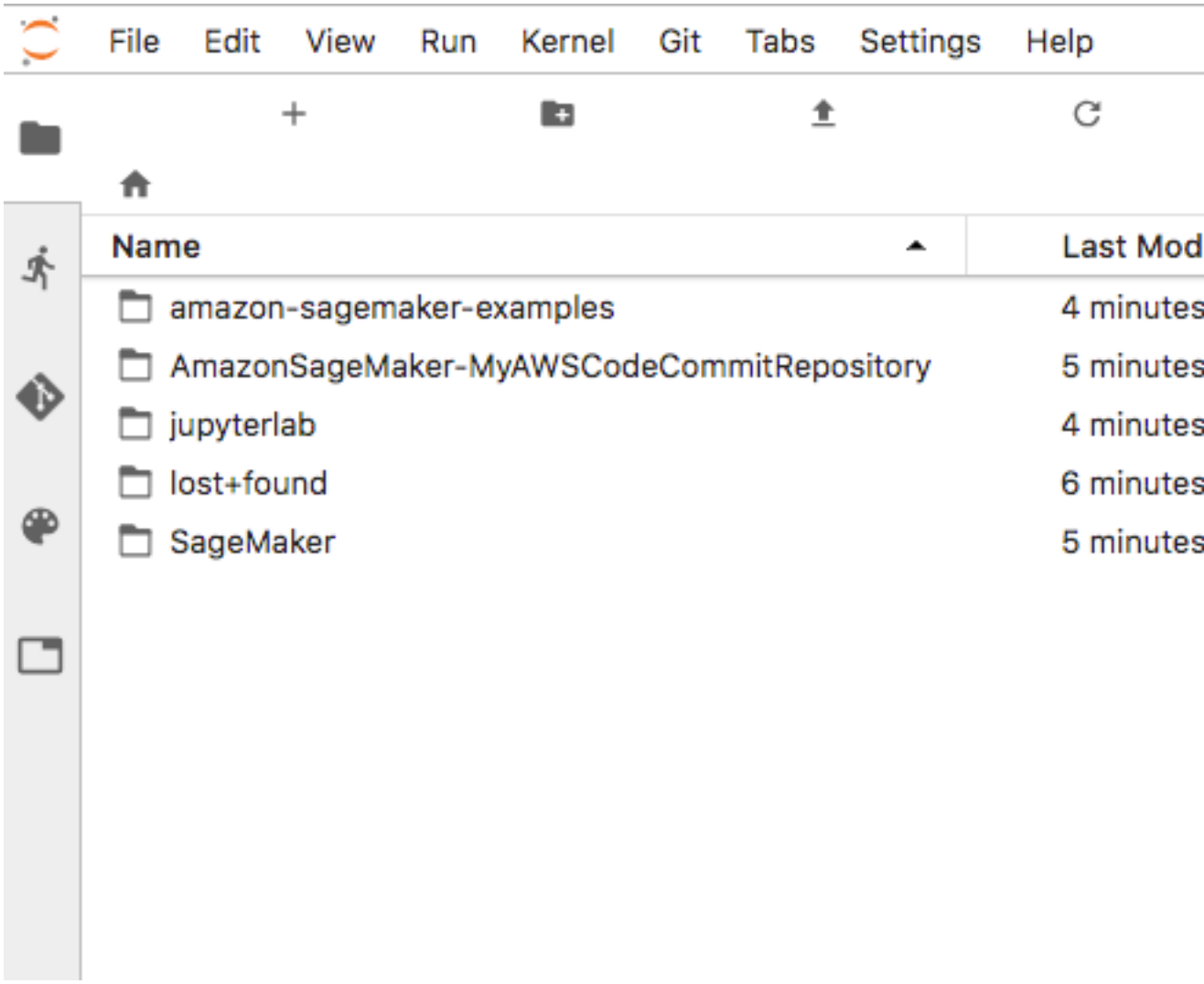
When you open a notebook instance that has Git repositories associated with it, it opens in the default repository, which is installed in your notebook instance directly under `/home/ec2-user/SageMaker`. You can open and create notebooks, and you can manually run Git commands in a notebook cell. For example:

```
!git pull origin master
```

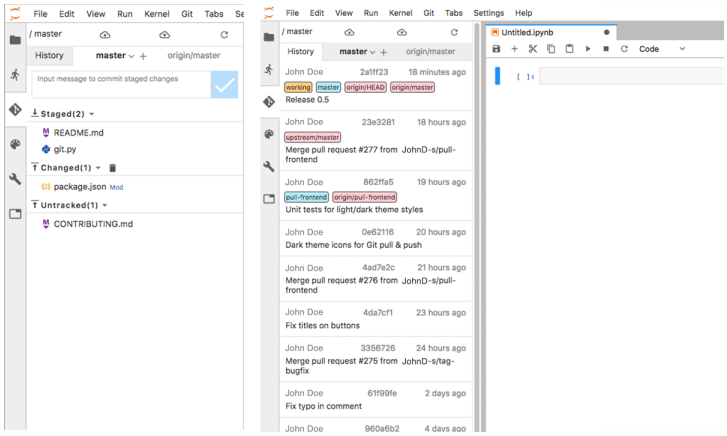
To open any of the additional repositories, navigate up one folder. The additional repositories are also installed as directories under `/home/ec2-user/SageMaker`.

If you open the notebook instance with a JupyterLab interface, the jupyter-git extension is installed and available to use. For information about the jupyter-git extension for JupyterLab, see https://github.com/jupyterlab/jupyterlab-git.

When you open a notebook instance in JupyterLab, you see the git repositories associated with it on the left menu:



You can use the jupyter-git extension to manage git visually, instead of using the command line:



Get Notebook Instance Metadata

When you create a notebook instance, Amazon SageMaker creates a JSON file on the instance at the location `/opt/ml/metadata/resource-metadata.json` that contains the `ResourceName` and `ResourceArn` of the notebook instance. You can access this metadata from anywhere within the notebook instance, including in lifecycle configurations. For information about notebook instance lifecycle configurations, see Customize a Notebook Instance (p. 40).

The `resource-metadata.json` file has the following structure:

```
{
    "ResourceArn": "NotebookInstanceArn",
    "ResourceName": "NotebookInstanceName"
}
```

You can use this metadata from within the notebook instance to get other information about the notebook instance. For example, the following commands get the tags associated with the notebook instance:

```
NOTEBOOK_ARN=$(jq '.ResourceArn'
            /opt/ml/metadata/resource-metadata.json --raw-output)
aws sagemaker list-tags --resource-arn $NOTEBOOK_ARN
```

The out put looks like the following:

```
{
    "Tags": [
        {
            "Key": "test",
            "Value": "true"
        }
    ]
}
```

Monitor Jupyter Logs in Amazon CloudWatch Logs

Jupyter logs include important information such as events, metrics, and health information that provide actionable insights when running Amazon SageMaker notebooks. By importing Jupyter logs into CloudWatch Logs, customers can use CloudWatch Logs to detect anomalous behaviors, set alarms, and discover insights to keep the Amazon SageMaker notebooks running more smoothly. You can access the logs even when the Amazon EC2 instance that hosts the notebook is unresponsive, and use the logs to troubleshoot the unresponsive notebook. Sensitive information such as AWS account IDs, secret keys, and authentication tokens in presigned URLs are removed so that customers can share logs without leaking private information.

To view Jupyter logs for a notebook instance:

1. Sign in to the AWS Management Console and open the Amazon SageMaker console at https://console.aws.amazon.com/sagemaker/.
2. Choose **Notebook instances**.
3. In the list of notebook instances, choose the notebook instance for which you want to view Jupyter logs.
4. Under **Monitor** on the notebook instance details page, choose **View logs**.

5. In the CloudWatch console, choose the log stream for your notebook intance. Its name is in the form `NotebookInstanceName/jupyter.log`.

For more information about monitoring CloudWatch logs for Amazon SageMaker, see Log Amazon SageMaker Events with Amazon CloudWatch (p. 466).